

Automated Multi-Site Infrastructure: Ansible API
Orchestration, WireGuard VPN Overlay, Dynamic
BGP Routing, and Centralized Identity Control

2026

Author: Rodríguez Rubén

INDICE:

INTRODUCTION	4
Key Objectives	4
Hardware Resource Allocation	5
Step 1. Environment initialization & Connectivity Control	6
1.1 Pre-Flight Connectivity Control & Routing Table Diagnostics	7
1.1.1. Multi-Site Gateway ICMP Reachability Test	7
1.1.2. Active Routing State Inspection (FreeBSD Kernel Table)	7
1.2 Outbound NAT & ISP Transit Verification	8
1.2.1. Outbound Internet Path Trace Verification	8
Step 2. Control Plane Establishment: Ansible & OPNsense API Integration	9
2.1 SSH Key-Based Cryptographic Trust Setup	9
2.1.1. Keypair Generation on Admin_Client (Ubuntu)	9
2.1.2. Public Key Distribution to OPNsense Nodes	9
2.2 Infrastructure Hardening: RBAC & Principle of Least Privilege (PoLP)	10
2.2.1. Granular Privilege Delegation Matrix	11
2.2.2. Attack Surface Reduction Analysis	12
2.3 Enterprise Directory Integration: Centralized Identity & Management	13
2.3.1 DNS Configuration	16
2.3.2 Organizational Unit (OU) & Active Directory Object Hierarchy	16
2.3.3 Centralized AAA via Secure LDAP (LDAPS)	19
2.3.4 OPNsense Integration & Certificate Import	20
2.3.5 Opnsense with LDAP authentication	22
2.4 Programmatic Control Plane Validation & Native REST API Orchestration	25
2.4.1. Pre-Deployment State: Firewall Rulebase Baseline	25
2.4.2. Automated Execution Log & Declarative Rule Insertion	25
2.4.3. Post-Deployment State: API Verification & Filter Reload State	26
3.1 WireGuard VPN Mesh Overlay Topology	26
3.1.1. Cryptographic Keypair Generation & Interface Binding via Ansible	27
3.1.2. Declarative Interface Binding & Routing Table Matrix	29
3.1.3. Execution Output & Cryptographic Handshake Verification	29
3.1.3.1 Peer Session State Audit (wg show)	29

3.1.3.2. Interface Assignment & Kernel Binding	31
3.1.3.3. Inter-Site ICMP Reachability Validation	32
Step 4. Dynamic Routing & Policy Enforcement (HQ ↔ Sede B)	32
4.1 Dynamic Routing Engine (FRRouting / BGP Implementation)	33
4.1.1 BGP Autonomous System (AS) Allocation & Peering Matrix	33
4.1.2 BGP Session Establishment & Peering Audit	35
4.1.3 Route Propagation & Kernel Inspection	36
4.2 Site-to-Site Inter-LAN Firewall Policy Enforcement	37
4.2.1 Allowed Services & Inter-Site Traffic Matrix	37
Step 5. End-to-End Enterprise Integration & Final Acceptance Testing	39
CONCLUSION	41

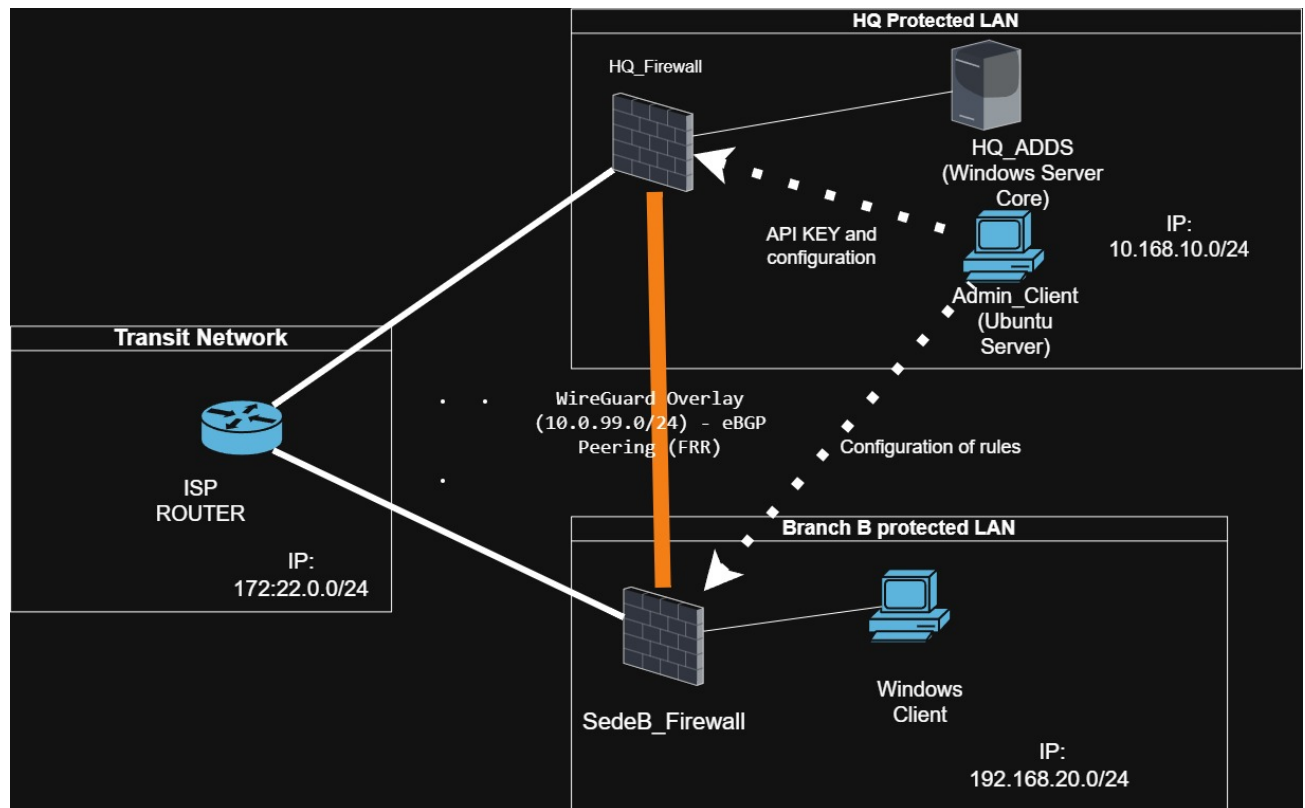
INTRODUCTION

This project addresses the design, implementation, and automation of a secure, interconnected network architecture between a central headquarters (**HQ**) and a remote branch (**Branch B**). To achieve a scalable and reproducible environment, the solution integrates **OPNsense** firewalls with **Infrastructure as Code (IaC)** principles using **Ansible**.

Key Objectives

- **Secure Overlay Network:** Establish an encrypted point-to-point tunnel using **WireGuard** to ensure efficient and confidential site-to-site communication.
- **Dynamic Routing:** Enable automatic route propagation between sites via **eBGP** (using the **FRR** routing suite).
- **Full Automation:** Provision and manage the entire infrastructure through Ansible playbooks interacting directly with the **OPNsense REST API**, eliminating manual tasks and hardcoded configurations.
- **Least Privilege Security:** Apply strict firewall policies based on the Principle of Least Privilege (**PoLP**) to secure management and data planes.

Figure 1: Architectural overview and laboratory topology of the project



Step 0. Preparation.

Before deploying the laboratory, it is essential to outline the technical prerequisites and resource allocations. The entire environment is virtualized utilizing a Type-2 Hypervisor, specifically **VMware Workstation Pro**.

To ensure optimal performance while running the target applications alongside kernel-level security monitoring tools, the host machine and the target Virtual Machine (VM) must meet specific hardware specifications.

Hardware Resource Allocation

- **Host Machine Requirements:** A robust multi-core processor (e.g., AMD Ryzen 5/7 or Intel i5/i7) and a minimum of 16 GB of RAM (32 GB recommended) to handle virtualization overhead smoothly.

Virtual Machine	RAM	vCPUs	Network Connections (VMware Switches)	Primary Purpose
HQ_Firewall	2 GB	2	ISP_Backbone, HQ_internal	Core Edge Gateway & BGP Router (Hub VPN)
SedeB_Firewall	2 GB	2	ISP_Backbone, SEDE_B	Branch B Edge Gateway & BGP Router (Spoke B)
ISP_Router	2 GB	2	NAT (VMware), ISP_Backbone	Simulated Transit Provider & WAN NAT Gateway
Admin_Client (Ubuntu)	4 GB	2	HQ_internal	Ansible Control Node & Central Management
HQ_ADDS (Win Server Core)	2 GB	2	HQ_internal	Active Directory Domain Services, LDAPS & DNS Server

IMPORTANT: This documentation focuses exclusively on the technical implementation of the project and does not cover the standard virtual machine creation wizard procedure.

Step 1. Environment initialization & Connectivity Control

To support a multi-site network architecture without address space leakage or external DHCP interference, the VMware Workstation virtual network interfaces must be bound to dedicated, isolated host-only virtual switches (VMnets). This ensures complete isolation of the routing segments before establishing the overlay VPN network.

The virtual networks are configured via the VMware **Virtual Network Editor** with the following structural parameters:

Virtual Switch	Name in Hypervisor	Subnet Segment	DHCP Status	Role in Topology
VMnet2	ISP_Backbone	172.22.0.0/24	Disabled	Public Transit Network (Simulated Internet)
VMnet3	HQ_internal	10.168.10.0/24	Disabled	Protected Headquarters LAN
VMnet4	SEDE_B	192.168.20.0/24	Disabled	Protected Remote Branch B LAN

1.1 Pre-Flight Connectivity Control & Routing Table Diagnostics

Before establishing cryptographic overlays (WireGuard) or provisioning dynamic routing protocols (BGP), the underlying transport layer (Layer 3) must be fully verified. This stage establishes the performance baseline and routing integrity of the simulated ISP backbone.

1.1.1. Multi-Site Gateway ICMP Reachability Test

From the **SedeB_Firewall** (192.168.20.X) , a series of ICMP echo requests are dispatched toward its local gateway and the core transit router to verify correct packet propagation across the hypervisor switches:

```
root@SedeB:~ # ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1): 56 data bytes
64 bytes from 192.168.20.1: icmp_seq=0 ttl=64 time=0.285 ms
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.278 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=0.316 ms
64 bytes from 192.168.20.1: icmp_seq=3 ttl=64 time=0.239 ms
64 bytes from 192.168.20.1: icmp_seq=4 ttl=64 time=0.289 ms
64 bytes from 192.168.20.1: icmp_seq=5 ttl=64 time=0.090 ms
^C
--- 192.168.20.1 ping statistics ---
6 packets transmitted, 6 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.090/0.250/0.316/0.075 ms
root@SedeB:~ #
```

1.1.2. Active Routing State Inspection (FreeBSD Kernel Table)

To ensure the edge firewalls correctly forward untunnelled external traffic through the transit network, we inspect the active IPv4 kernel routing table of the OPNsense nodes using low-level FreeBSD diagnostic commands:

```
round-trip min/avg/max/stddev = 0.090/0.250/0.316/0.075 ms
root@SedeB:~ # netstat -rn -f inet
Routing tables

Internet:
Destination      Gateway          Flags           Netif Expire
default          172.22.0.1      UGS             em0
127.0.0.1        link#3          UH              lo0
172.22.0.0/24    link#1          U               em0
172.22.0.188     link#3          UHS             lo0
192.168.20.0/24  link#2          U               em1
192.168.20.1     link#3          UHS             lo0
root@SedeB:~ #
```

Technical Interpretation of Routing Flags:

- U (Usable): The route is active and dynamically accessible.
- G (Gateway): Indicates that the destination is reached indirectly through an intermediate gateway (172.22.0.1).
- s (Static): Denotes that the route was statically bound during the initial system initialization phase.

1.2 Outbound NAT & ISP Transit Verification

To fully simulate a production-grade enterprise environment, the edge nodes must not only establish inter-site connectivity but also reach external WAN resources (such as public software repositories, package updates, or external telemetry collectors) through the ISP_Router. This egress path requires active **Dynamic Outbound NAT (Masquerading)** on the simulated transit gateway.

1.2.1. Outbound Internet Path Trace Verification

From the HQ_Firewall console, we execute a drill (nslookup in FreeBSD) targeting a public destination to verify that packets successfully transit through the simulated ISP infrastructure, undergo NAT translation, and egress the virtual hypervisor fabric:

```
root@HQ-Edge:~ # drill www.google.es
;; ->>HEADER<<- opcode: QUERY, rcode: NOERROR, id: 2417
;; flags: qr rd ra ; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 3
;; QUESTION SECTION:
;; www.google.es.      IN      A

;; ANSWER SECTION:
www.google.es. 17      IN      A      172.217.22.67

;; AUTHORITY SECTION:

;; ADDITIONAL SECTION:
inf.portmaster. 0      IN      TXT      "accepted: allowing dns request"
inf.portmaster. 0      IN      TXT      "served from cache, resolved by 9.9.9.9
(dns://9.9.9.9:53#system)"
inf.portmaster. 0      IN      TXT      "record valid for 1m55s"

;; Query time: 8 msec
;; SERVER: 9.9.9.9
;; WHEN: Wed Jul 15 19:28:01 2026
;; MSG SIZE rcvd: 257
root@HQ-Edge:~ #
```

Step 2. Control Plane Establishment: Ansible & OPNsense API Integration

To achieve absolute orchestration and guarantee that all security policies, interface definitions, and VPN tunnels are repeatable and immune to manual administrative errors, a secure programmatic control plane must be established. This plane links the **Admin_Client (Ubuntu)** with the OPNsense cluster (HQ_Firewall, SedeB_Firewall).

This control channel utilizes two core mechanisms: **SSH Key-Based Cryptographic Trust** for low-level bootstrapping, and the native **OPNsense Core REST API** for structured service configuration and deployment.

2.1 SSH Key-Based Cryptographic Trust Setup

To prevent administrative password exposure over the transit network and allow Ansible to execute high-privilege tasks seamlessly, we enforce key-based authentication utilizing a modern elliptic-curve standard (Ed25519).

2.1.1. Keypair Generation on Admin_Client (Ubuntu)

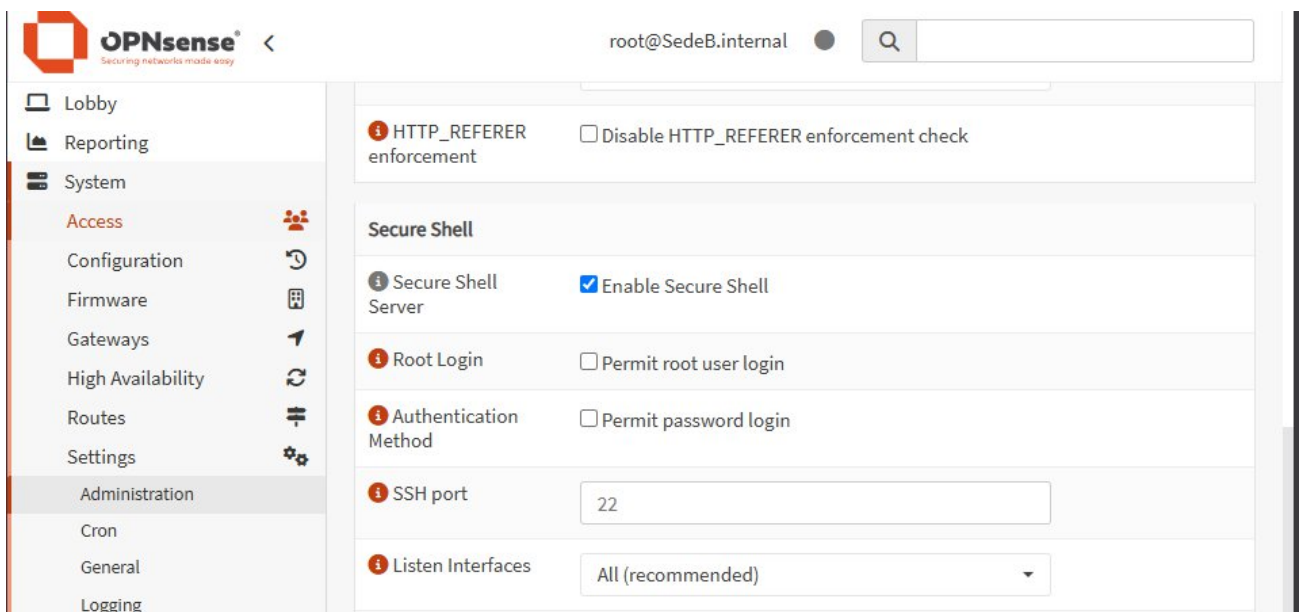
From the command line of the Admin_Client, generate an administrative Ed25519 keypair:

```
admins@admins-VMware-Virtual-Platform:~$ ssh-keygen -t ed25519 -C "ansible-admin-control" -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/admins/.ssh/id_ed25519
Your public key has been saved in /home/admins/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:rB9wGPvJWyTweUoVaQ2c9K08VB+WX0LQ0gn51k+D6Ag ansible-admin-control
The key's randomart image is:
+--[ED25519 256]--+
|      o+=o*.. |
|      =o+.+. |
| o   .. =o*.. |
| E + + =o=+ |
| + S O   ...+ |
| B O o       |
| . * o       |
| . +        |
| o          |
+-----[SHA256]-----+
```

2.1.2. Public Key Distribution to OPNsense Nodes

Using the temporary root credentials enabled during the console bootstrap phase, the public key is copied to each edge gateway:

First, before do the steps for public key distribution, we should be sure if the openssh service is work, we will continue with the step, if not, you should check if the service is up.



```
root@SedeB:~ # service openssh status
openssh is running as pid 82179.
root@SedeB:~ #
```

2.2 Infrastructure Hardening: RBAC & Principle of Least Privilege (PoLP)

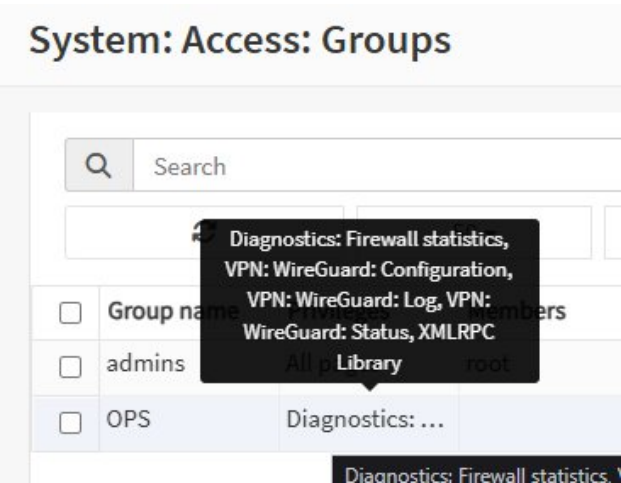
To achieve a hardened security posture and minimize the attack surface of the perimeter gateways, utilizing the default root account or assigning global admins group privileges to automation tools is strictly rejected. This architecture enforces strict **Role-Based Access Control (RBAC)** by provisioning a highly restricted service account and a custom privilege group.

2.2.1. Granular Privilege Delegation Matrix

A dedicated privilege group named Automation_Ops is constructed on each OPNsense node. This group is explicitly restricted from modifying global system access, changing user databases, or altering main cryptographic system certificates. Instead, it is granted only the exact system libraries and service structures required for remote API orchestration:

Service Account	Assigned RBAC Group	OS Shell	Explicit API Privileges Granted
ansible-admin	Automation_Ops	/bin/sh	1. Diagnostics: Firewall statistics (System state validation) 2. VPN: WireGuard: Configuration (Tunnel orchestration) 3. VPN: WireGuard: Log & Status (Operational metrics) 4. XMLRPC Library (Core API transit communication)

The following active state snapshot confirms the precise cryptographic privilege boundary configured within the user access manager interface:



2.2.2. Attack Surface Reduction Analysis

By decoupling the ansible-admin account from the unrestricted root superuser, any potential programmatic credential compromise is strictly contained within the VPN and diagnostic sub-systems. This enforces a state of *Defense-in-Depth* within our management plane, ensuring that even under an orchestration-compromise scenario, the core system integrity and local authentication files remain completely isolated.

System: Access: Users

Users

ApiKeys

50

☐	Username	Group membership	Full name
☐	 root	admins	System Administrator
☐	 ansible	Domain_Admin	

Preferred landing page

Language

Default

Login shell

/bin/sh

Expiration date

Group membership

admins

Clear All

Select All

Privileges

Nothing selected

Clear All

Select All

OTP seed

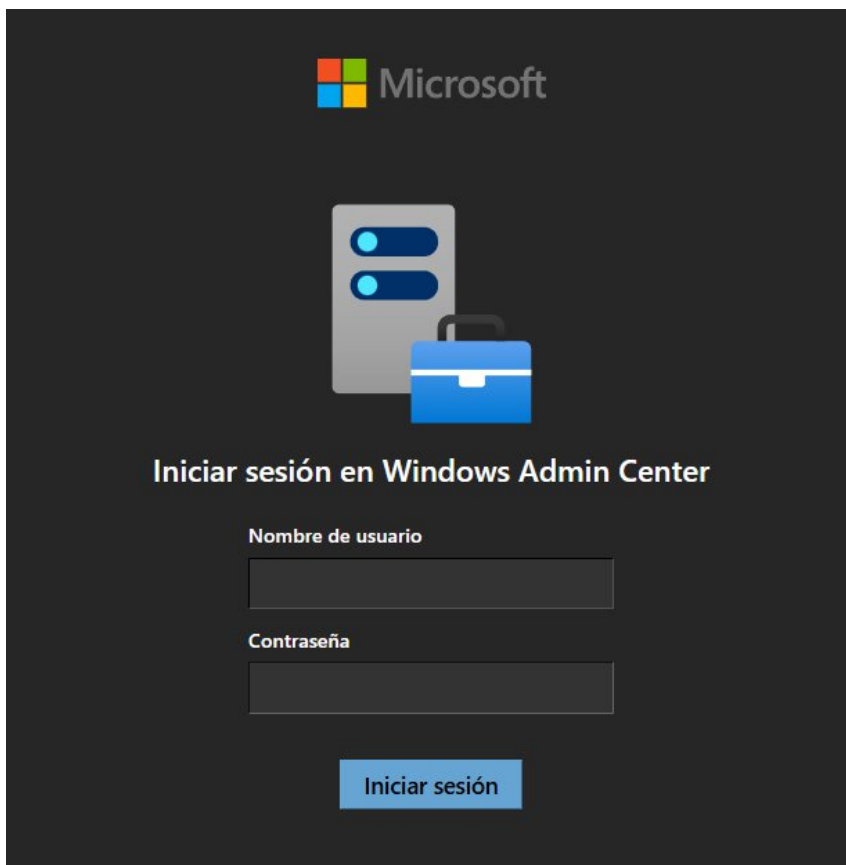
show

Authorized Keys

2.3 Enterprise Directory Integration: Centralized Identity & Management

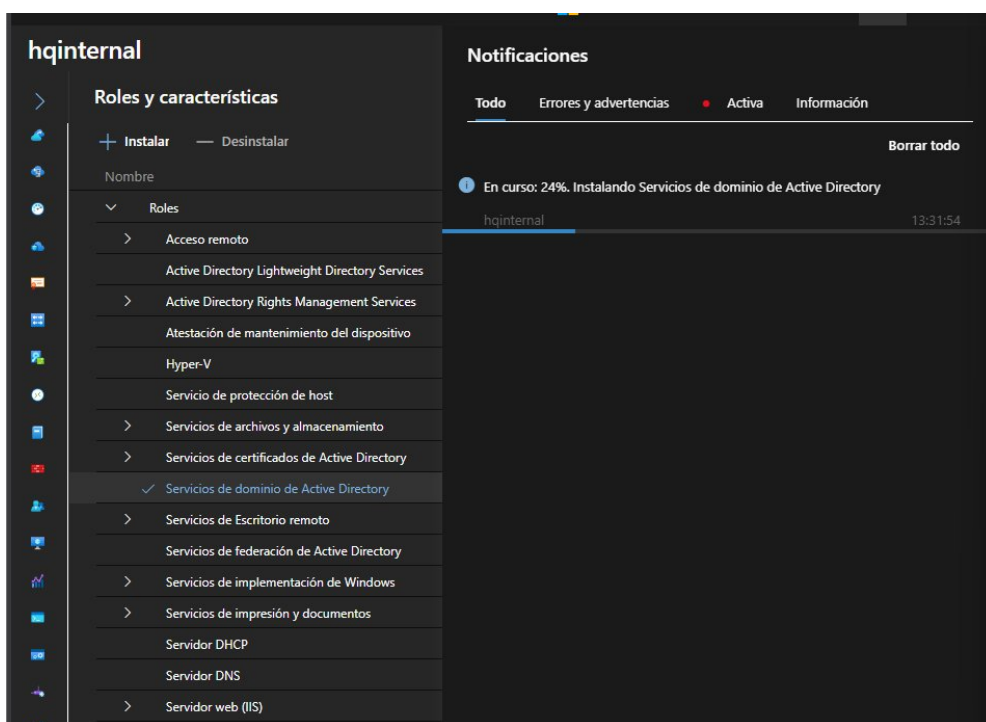
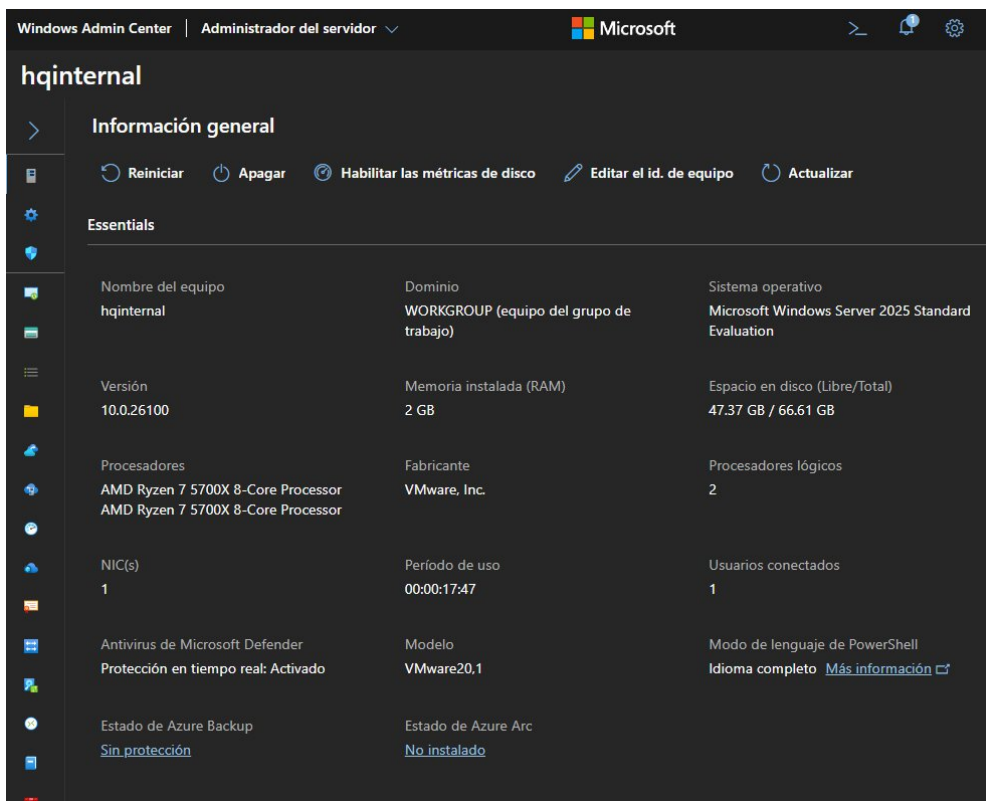
To professionalize the IAM architecture, administrative authentication was centralized by integrating an Active Directory Domain Services (AD DS) controller, eliminating local credential databases across the gateways.

- Hardened Domain Controller (Windows Server Core): Deployed as 'HQ_ADDS' (2 vCPUs, 1.5 GB RAM) within 'HQ_internal'. Operating without a GUI minimizes resource overhead (saving ~2 GB of physical RAM) and strictly reduces the local attack surface.
- Decoupled Management (Windows Admin Center): Administered remotely from the 'Admin_Client' via a secure web-based HTTPS gateway, enabling full object and certificate management without local interactive console sessions.
- Centralized AAA (LDAPS): The three OPNsense gateways offload administrative SSH authentication to the AD DS controller. Communications are encrypted via LDAP over SSL (LDAPS, TCP port 636) to prevent credential sniffing and ensure unified user lifecycle management.



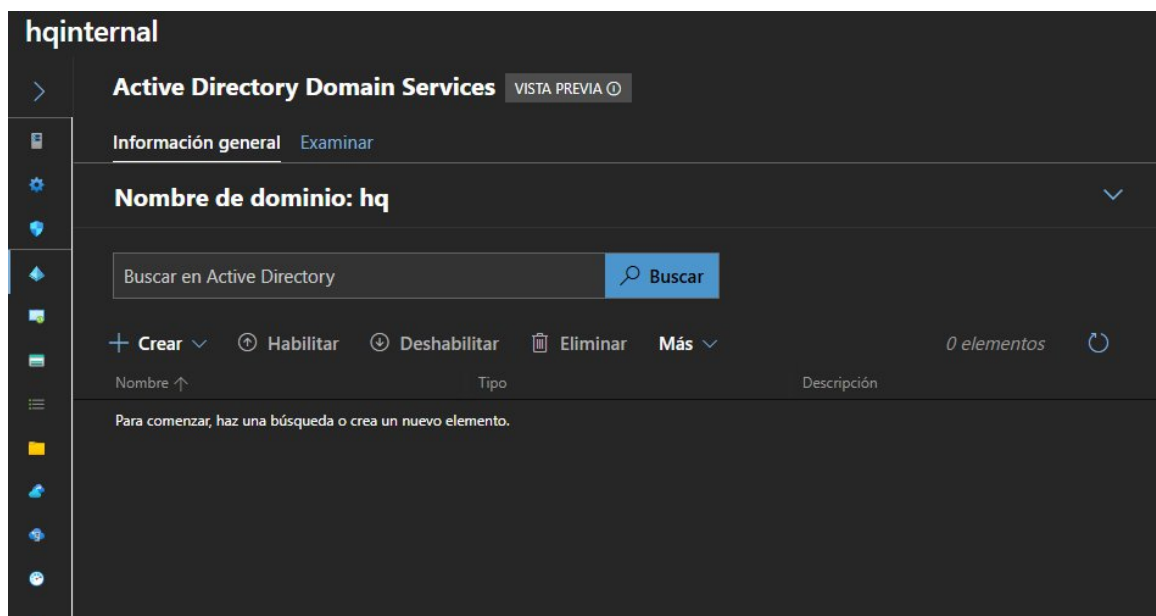
When we have connection with the Windows Server Core through WAC (Windows Admin center into the Server installed).

We can do any thing in the server, in this case, we are going to install the role of Active Directory.



After we watched the command execution was successfully, we can administrate all through the WAC.

Here the example.



2.3.1 DNS Configuration.

It is crucial to implement a reliable DNS service within our infrastructure, as DNS plays a vital role in client resolution and overall architectural stability.

```
PS C:\Windows\System32> Add-DnsServerForwarder -IPAddress "10.168.10.1" -PassThru

UseRootHint      : True
Timeout(s)       : 3
EnableReordering : True
IPAddress         : {10.168.10.1, 9.9.9.9, 8.8.8.8}
ReorderedIPAddress : {10.168.10.1, 9.9.9.9, 8.8.8.8}

PS C:\Windows\System32>
```

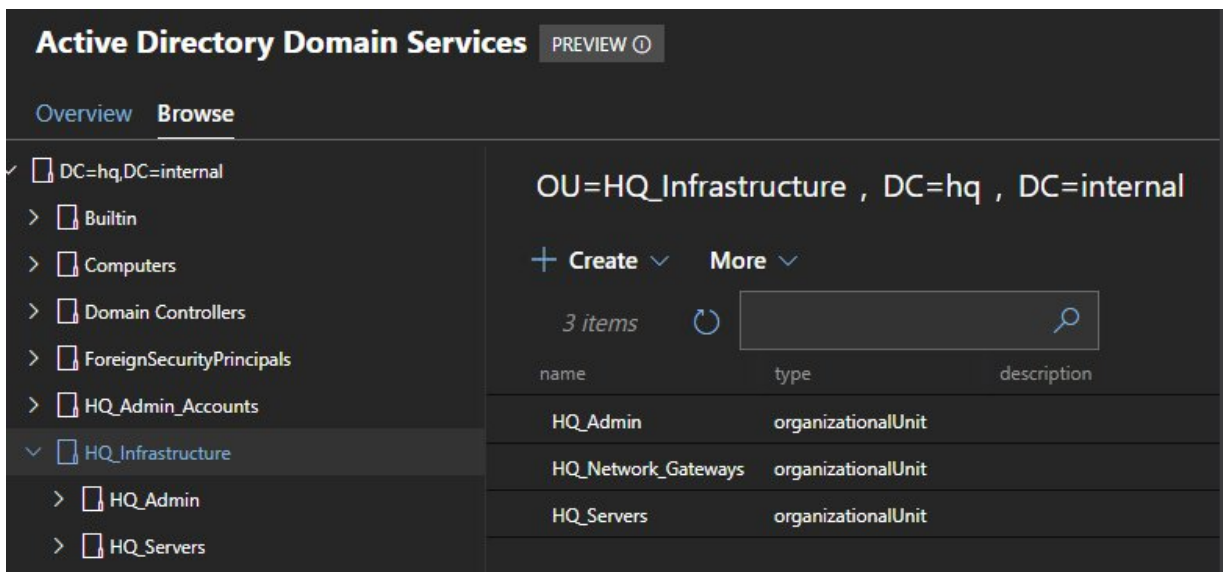
2.3.2 Organizational Unit (OU) & Active Directory Object Hierarchy

To enforce granular administrative boundaries, a structured hierarchy of Organizational Units (OUs) was established within the 'hq.internal' root, avoiding default generic containers:

Tiered OU Architecture: A parent OU named 'HQ_Infrastructure' centralizes three specialized sub-OUs:

- HQ_Admin_Accounts: For personalized administrative credentials.
- HQ_Servers: For domain-joined server computer objects.
- HQ_Network_Gateways: For perimeter assets and service accounts assigned to the three OPNsense nodes.

Identity Management & Privilege Separation: Dedicated administrative accounts within the 'HQ_Admin_Accounts' OU are nested into the 'Domain Admins' group, ensuring clear accountability and enforcing the principle of least privilege.



This command, is if you have problem for remove any OU which you wouldn't in the scheme.

In Active Directory Domain Services (AD DS) environments, Organizational Unit (OU) objects are created with the ProtectedFromAccidentalDeletion attribute enabled by default. This security mechanism prevents the unintended loss of container objects, Group Policy Objects (GPOs), and associated accounts.

Controlled Unprotection and Deletion Procedure: Executing automation scripts to unprotect and subsequently delete OUs via PowerShell must be strictly restricted to testing environments or scheduled decommissioning workflows governed by a formal Change Management process

In Active Directory Domain Services (AD DS) environments, Organizational Unit (OU) objects are created with the ProtectedFromAccidentalDeletion attribute enabled by default. This security mechanism prevents the unintended loss of container objects, Group Policy Objects (GPOs), and associated accounts.

Controlled Unprotection and Deletion Procedure: Executing automation scripts to unprotect and subsequently delete OUs via PowerShell must be strictly restricted to testing environments or scheduled decommissioning workflows governed by a formal Change Management process

```
PS C:\Windows\System32> # 1. Quitar el candado de protección de la OU suelta en la raíz
PS C:\Windows\System32> Set-ADOrganizationalUnit -Identity "OU=HQ_Admin_Accounts,DC=hq,DC=internal" -Protect
edFromAccidentalDeletion $false
PS C:\Windows\System32>
PS C:\Windows\System32> # 2. Eliminar la OU definitivamente
PS C:\Windows\System32> Remove-ADOrganizationalUnit -Identity "OU=HQ_Admin_Accounts,DC=hq,DC=internal" -Conf
irm:$false
```

When we have the OUs, it is very important have a admin user different, for Admins tasks, so I created one user for me to have just this user for do admins tasks.

DC=hq,DC=internal

- Builtin
- Computers
- Domain Controllers
- ForeignSecurityPrincipals
- HQ_Infrastructure
 - HQ_Admin
 - HQ_Network_Gateways
 - HQ_Servers

OU=HQ_Admin , OU=HQ_Infrastructure , DC=hq , DC=interi

+ Create ▾ ⬆ Enable ⬇ Disable More ▾

1 item ↻

name	type	description
Ruben	user	

And of course, it's very important to add the new user into the Domain Controller.

Active Directory Domain Services > Group properties: Admins. del dominio

Group + Add 🗑 Delete 2 items

Membership

Name ↑	Active Directory Domain Services Folder
Administrador	internal.hq.Users.Administrador
Ruben	internal.hq.HQ_Infrastructure.HQ_Admin.Ru...

2.3.3 Centralized AAA via Secure LDAP (LDAPS)

To secure administrative authentication across the infrastructure, the three OPNsense gateways offload AAA services to the Active Directory Domain Services (AD DS) controller. To prevent credential sniffing and cleartext transmission, communications are strictly encrypted using LDAP over SSL (LDAPS) via TCP port 636.

Roles and features			
+ Install — Uninstall		208 items	1 selected ×
Name	State	Type	
▼ Roles	4 of 81 Instal...		
➤ Acceso remoto	0 of 3 Installed	Role	
Active Directory Lightweight Directory Services	Available	Role	
➤ Active Directory Rights Management Services	0 of 2 Installed	Role	
Atestación de mantenimiento del dispositivo	Available	Role	
Hyper-V	Available	Role	
Servicio de protección de host	Available	Role	
➤ Servicios de archivos y almacenamiento	2 of 12 Instal...	Role	
▼ Servicios de certificados de Active Directory	0 of 6 Installed	Role	
✓ Entidad de certificación	Available	Role Service	
Inscripción web de entidad de certificación	Available	Role Service	
Respondedor en línea	Available	Role Service	
Servicio de inscripción de dispositivos de red	Available	Role Service	
Servicio web de directiva de inscripción de certificados	Available	Role Service	

```
PS C:\Windows\System32> Get-WindowsFeature AD-Certificate

Display Name                    Name                Install State
-----
[X] Servicios de certificados de Active Directory  AD-Certificate      Installed

PS C:\Windows\System32>
```

Then, we install the authority in our AD Server, and we checked if all it's works.

```
PS C:\Windows\System32> Install-AdcsCertificationAuthority -CAType EnterpriseRootCA -CryptoProviderName "RSA#Microsoft Software Key Storage Provider" -KeyLength 2048 -HashAlgorithmName SHA256 -Force

ErrorId ErrorString
-----
0

PS C:\Windows\System32> Get-NetTCPConnection -LocalPort 636 -State Listen

LocalAddress      LocalPort RemoteAddress      RemotePort State      AppliedSetting
-----
::: 636 ::: 0 Listen
0.0.0.0 636 0.0.0.0 0 Listen

PS C:\Windows\System32>
```

This step is very important if we want download the certificate, for example, this certificate for authorities in the Opnsense.

```
PS C:\Windows\System32> Get-ChildItem Cert:\LocalMachine\Root | Where-Object {$_.Subject -like "*HQ_ADDS*"} | Export-Certificate -FilePath C:\ca_root_hq.cer

Directorio: C:\

Mode                LastWriteTime         Length Name
----                -
-a-----         17/07/2026   18:10             20 ca_root_hq.cer

PS C:\Windows\System32>
```

	ca_root_hq.cer	17/07/2026, 18:10:38	File	1 KB
	DumpStack.log.tmp	17/07/2026, 17:25:31	File	12 KB
	pagefile.sys	17/07/2026, 17:25:31	File	1,179,648 KB

2.3.4 OPNsense Integration & Certificate Import

To establish trust between the security gateways and the Server Core directory, the root certificate generated by the Enterprise CA must be imported into each OPNsense appliance. This configuration shifts authentication from the local user database to the centralized directory infrastructure over an encrypted channel.

```
PS C:\Windows\System32> $cert = Get-ChildItem Cert:\LocalMachine\Root | Select-Object -First 1
PS C:\Windows\System32> [System.IO.File]::WriteAllText("C:\ca_root_hq.cer", "-----BEGIN CERTIFICATE-----`r`n" + [System.Convert]::ToBase64String($cert.RawData, [System.Base64FormattingOptions]::InsertLineBreaks) + "`r`n-----END CERTIFICATE-----")
PS C:\Windows\System32>
```

	ca_root_hq.cer	17/07/2026, 18:...	File	2 KB
---	----------------	--------------------	------	------

Edit Certificate

Method Import an existing Certificate Authority

Description HA_Internal_CA

Output (PEM format)

Certificate data -----BEGIN CERTIFICATE-----

When we import the certificate, we have the authority in our Opnsense

 **OPNsense** Securing networks made easy

root@HQ-Edge.empresa.local

Lobby

Reporting

System

Access

Configuration

Firmware

Gateways

High Availability

Routes

Settings

Snapshots

Trust

Authorities

System: Trust: Authorities

Certificates

Description

Issuer

Name

Usages

Valid from

Valid to

Commands

HA_Internal...

/DC=interna...

0

Jul 17, 2026 ...

Jul 17, 2031 ...

«

<

1

>

»

Showing 1 to 1 of 1 entries

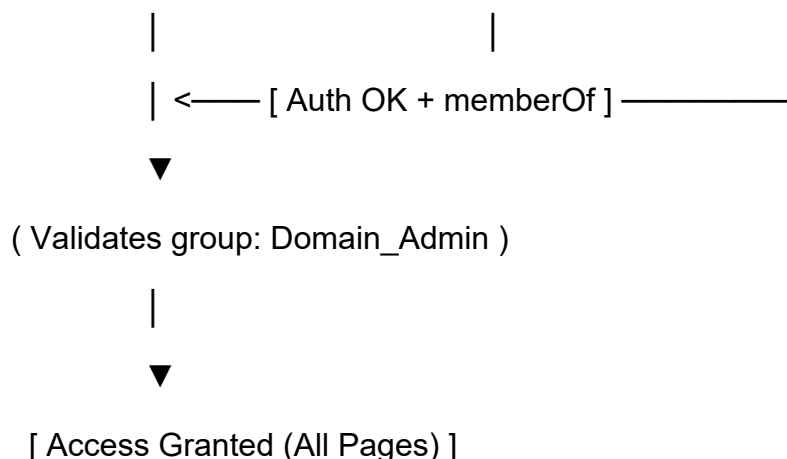
+

2.3.5 Opnsense with LDAP authentication.

To centralize identity management across the virtualized infrastructure, OPNsense is integrated with the Active Directory Domain Services running on Windows Server Core. This mechanism delegates credential validation (authentication) and privilege assignment (authorization) to the domain controller securely via LDAPS.

Operational Workflow

[User: ruben] —(Web Login)—> [OPNsense] —(LDAPS / 636)—> [Windows Server Core]



- **Authentication (Validation):** The firewall acts as an LDAP client. When a user submits their credentials via the web UI, OPNsense queries the server in real-time matching the samAccountName attribute.
- **Authorization (Mirror Groups):** Upon successful authentication, OPNsense extracts the user's memberOf attribute. The firewall does not clone the Active Directory user database; instead, it relies on a **local mirror group** inside OPNsense (e.g., Domain_Admin) matching the exact group name returned by the domain. System privileges (such as All pages) are mapped to this local group.
- **Just-in-Time (JIT) Provisioning:** With the *Automatic user creation* directive enabled, OPNsense dynamically generates the local user profile during their first successful login, binding them to the synchronized group infrastructure.

Fallback Safeguard: Keeping the *Local Database* enabled as a secondary method prevents a complete administrative lockout in the event that the Windows Server Core virtual machine is offline, unreachable, or undergoing maintenance, allowing emergency access via the local root account.

Firstly, if we would like to have centralized the users of AD into the Firewall, we should configure some parts of Opnsense, this is only for Administration of Firewall of course.

Descriptive name	HA_ADDS
Type	LDAP
Hostname or IP address	hqinternal.hq.internal
Port value	636
Transport	SSL - Encrypted
Protocol version	3
Bind credentials	User DN: Ruben@hq.internal Password: *****
Search scope	Entire Subtree
Base DN	DC=hq,DC=internal
Authentication containers	OU=HQ_Admin,OU=HQ_Infrastructure,DC=hq,D... Select
Extended Query	
User naming attribute	sAMAccountName

When Opnsense detected the Database of AD, we can choose which Unit organization we would like to synchronized.

Please select which containers to Authenticate against:

- ☐ OU=Domain Controllers,DC=hq,DC=internal
- ☒ OU=HQ_Admin,OU=HQ_Infrastructure,DC=hq,DC=internal
- ☐ OU=HQ_Infrastructure,DC=hq,DC=internal
- ☐ OU=HQ_Network_Gateways,OU=HQ_Infrastructure,DC=hq,DC=internal
- ☐ OU=HQ_Servers,OU=HQ_Infrastructure,DC=hq,DC=internal
- ☐ CN=Users,DC=hq,DC=internal

The test of as Opnsense ask to AD about the user we tested.

Authentication Server	HA_ADDS
Username	ruben
Password	
Test	

System: Access: Tester

User: ruben authenticated successfully.

This user is a member of these groups:
Domain Admin

May access the following locations, depending on source address:

Uri Networks

Attributes received from server:

```

attributes received from server:
dn => CN=ruben,OU=HQ_Admin,OU=HQ_Infrastructure,DC=hq,DC=internal
objectclass => top
person
organizationalPerson
user
cn => ruben
givenname => ruben
distinguishedname => CN=ruben,OU=HQ_Admin,OU=HQ_Infrastructure,DC=hq,DC=internal
instancetype => 4
whencreated => 20260717180441.0Z
whenchanged => 20260717180647.0Z
usncreated => 16591
memberof => CN=Admins. del dominio,CN=Users,DC=hq,DC=internal
usnchanged => 16599
name => ruben
objectguid => 00000000-0000-0000-0000-000000000000
useraccountcontrol => 512
badpwdcount => 0
codepage => 0
countrycode => 0
badpasswordtime => 0
lastlogoff => 0
lastlogon => 0
pwdlastset => 134287850812166439
primarygroupid => 513
objectsid => 00000000:0000,00P000PQ0
accountexpires => 9223372036854775807
logoncount => 0

```

Important: The server of Authentication it could be between Local and HA_ADDS (AD virtual machine Windows), as security if some part failed, we can always login into the firewall.

Authentication

 Server

Local Database, HA_ADDS

 Sudo

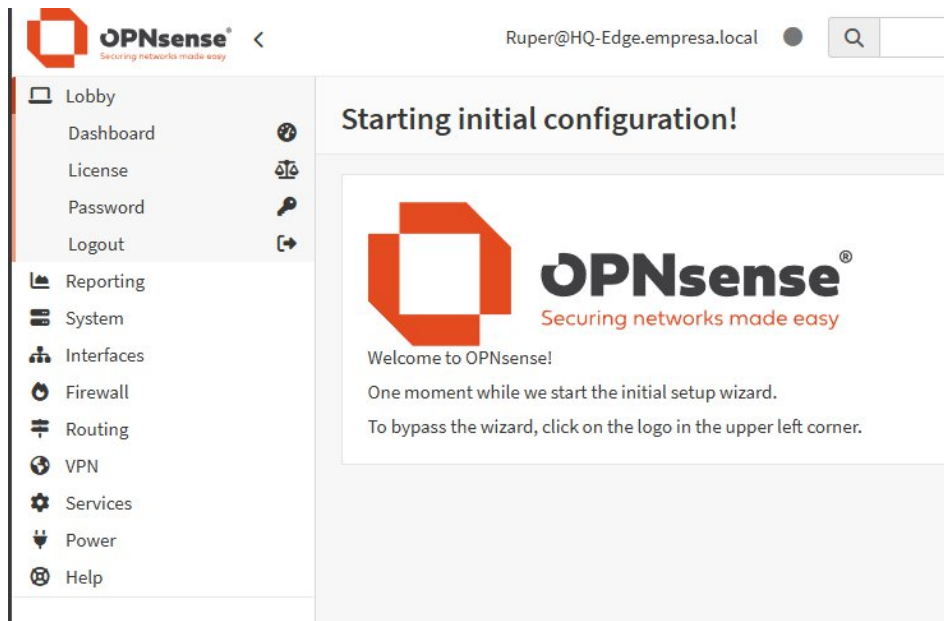
Disallow

wheel

 User OTP seed

Nothing selected

We can login with our AD user in Opnsense.



2.4 Programmatic Control Plane Validation & Native REST API Orchestration

To validate the operational integrity of the REST API control plane before deploying complex multi-site networking structures (such as WireGuard interfaces), a programmatic smoke test is executed from the Admin_Client. This test performs a live declarative state change by injecting a unified firewall rule for network telemetry/FTP transit directly into the OPNsense packet filter engine.

2.4.1. Pre-Deployment State: Firewall Rulebase Baseline

Before transmitting payload states via Ansible, the active rulebase on the perimeter firewalls is audited via the graphical user interface (GUI) to establish a clean baseline and verify that no conflicting or manual placeholder rules exist on the target interface.

2.4.2. Automated Execution Log & Declarative Rule Insertion

The declarative policy defined in prue.yml is executed from the Ansible control node. This orchestration bypasses third-party legacy abstractions, communicating natively with the /api/firewall/filter/add_rule endpoint via encrypted HTTPS.

The task payload enforces basic authentication utilizing the credentials stored inside the cryptographic vault plane (opnsense_api_key / opnsense_api_secret).

Bash

```
ruben@adminsyt:~/proyecto$ ansible-playbook -i inventory/host.ini prue.yml --ask-vault-pass
```

```

ruben@adminsyst:~/proyecto$ ansible-playbook prue.yml --limit HQ_Firewall

PLAY [OPNsense API NATIVA - Crear Regla FTP] *****

TASK [Insertar regla FTP 21 en la sección unificada de OPNsense] *****
ok: [HQ_Firewall]

TASK [Mostrar respuesta de creación nativa] *****
ok: [HQ_Firewall] => {
  "msg": "Resultado de la API: {'result': 'saved', 'uuid': 'b4b8f655-feb7-4757-ad7a-f8abc6608646'}"
}

TASK [Aplicar cambios (Filter Reload)] *****
ok: [HQ_Firewall]

PLAY RECAP *****
HQ_Firewall : ok=3    changed=0    unreachable=0    failed=0    skipped
=0    rescued=0    ignored=0

ruben@adminsyst:~/proyecto$ █

```

2.4.3. Post-Deployment State: API Verification & Filter Reload State

To commit the newly injected rule from the volatile memory stack to the active packet filter kernel (pf), a secondary POST request is dispatched to `/api/firewall/filter/apply`. This programmatic trigger forces a filter reload without causing state table drops.

The final state configuration is verified directly within the OPNsense rule manager, confirming absolute synchronization between our programmatic control plane and the running infrastructure.

Step 3. Site-to-Site VPN Overlay & Dynamic Routing Control Plane (WireGuard & BGP)

With the control plane established via Ansible and the management identities centralized through LDAPS, the next phase focuses on building the secure data transit layer. This step establishes encrypted site-to-site overlay tunnels connecting the Headquarters (**HQ_Firewall**) with the remote branches (**SedeB_Firewall**) over the simulated untrusted public transit network (**ISP_Backbone**), followed by dynamic route propagation using the Border Gateway Protocol (BGP).

3.1 WireGuard VPN Mesh Overlay Topology

To support modern, high-throughput, low-latency cryptographic transport, a Hub-and-Spoke overlay topology is implemented utilizing the **WireGuard** protocol. WireGuard operates at the kernel level (FreeBSD) using state-of-the-art cryptography (ChaCha20-Poly1305 for encryption and BLAKE2s for hashing), providing superior performance over traditional IPsec suites.

3.1.1. Cryptographic Keypair Generation & Interface Binding via Ansible

For each edge node, unique asymmetric keypairs (Private/Public) must be generated to establish mutual cryptographic trust. The overlay addressing matrix isolates tunnel traffic inside a dedicated point-to-point transit segment:

- **HQ_Firewall (Hub):** Acts as the central concentration point listening on public WAN endpoint 172.22.0.X.
- **SedeB_Firewall (Spoke B):** Initiates persistent keep-alive handshakes towards HQ.

Here we have in the HQ_Firewall.yml the subnet of Wireguard for our infrastructure.

```
wg_tunnel_address: "10.0.99.1/24"
ruben@adminsyst:~/proyecto$
```

```
ruben@adminsyst:~/proyecto$ ansible-playbook deploy_wireguard.yml --ask-vault-pass
Vault password:

PLAY [OPNsense API NATIVA - Despliegue de Infraestructura WireGuard] *****
TASK [Generar clave privada WireGuard] *****
ok: [HQ_Firewall]
TASK [Derivar clave pública WireGuard] *****
ok: [HQ_Firewall]
TASK [Generar Instancia de Servidor WireGuard local] *****
ok: [HQ_Firewall]
TASK [Debug: Mostrar respuesta de la API de WireGuard] *****
ok: [HQ_Firewall] => {
  "msg": "Servidor creado en HQ_Firewall. Respuesta: {'result': 'saved', 'uuid': 'e7f49002-d2a0-471a-862a-eaf61b7fa6ab'}"
}
PLAY RECAP *****
HQ_Firewall : ok=4 changed=0 unreachable=0 failed=0 skipped
=0 rescued=0 ignored=0

ruben@adminsyst:~/proyecto$ ^C
ruben@adminsyst:~/proyecto$
```

When we executed the playbook, we should have in our Opnsense, the new configuration of WireGuard.

Lobby

Reporting

System

Interfaces

Firewall

VPN

IPsec

OpenVPN

WireGuard

Instances

Peers

Peer generator

Status

Log File

Services

Power

Help

VPN: WireGuard

Instances

Peers

Peer generator

Q

Search

↺

50 ▾

☰ ▾

🔗

☐	Enabled	Name	Instance	Listen port	Tunnel add...	Peers	Commands
☐	☒	WG_Overlay	wg0	51820	10.0.99.1/24		✎📄🗑

« < 1 > »

Showing 1 to 1 of 1 entries

+

☑

☐

🗑

full help 🔴

📘 Enable WireGuard

☐

Apply

VPN: WireGuard

Instances

Peers

Peer generator

Q

Search

↺

50 ▾

☰ ▾

🔗

☐	Enabled	Name	Instance	Listen port	Tunnel add...	Peers	Commands
☐	☒	WG_Overlay	wg0	51820	10.0.99.2/24		✎📄🗑

« < 1 > »

Showing 1 to 1 of 1 entries

+

☑

☐

🗑

full help 🔴

📘 Enable WireGuard

☐

3.1.2. Declarative Interface Binding & Routing Table Matrix

To isolate site-to-site transit traffic from public WAN paths, a dedicated point-to-point overlay addressing matrix is deployed across the virtualized nodes:

Node Role	Target Hostname	WAN Transit Endpoint	WireGuard Overlay IP	Local Protected Subnet
VPN Hub	HQ_Firewall	172.22.0.187:51820	10.0.99.1/24	10.168.10.0/24 (VMnet3)
Spoke B	SedeB_Firewall	172.22.0.188:42943	10.0.99.2/24	192.168.20.0/24 (VMnet4)

3.1.3. Execution Output & Cryptographic Handshake Verification

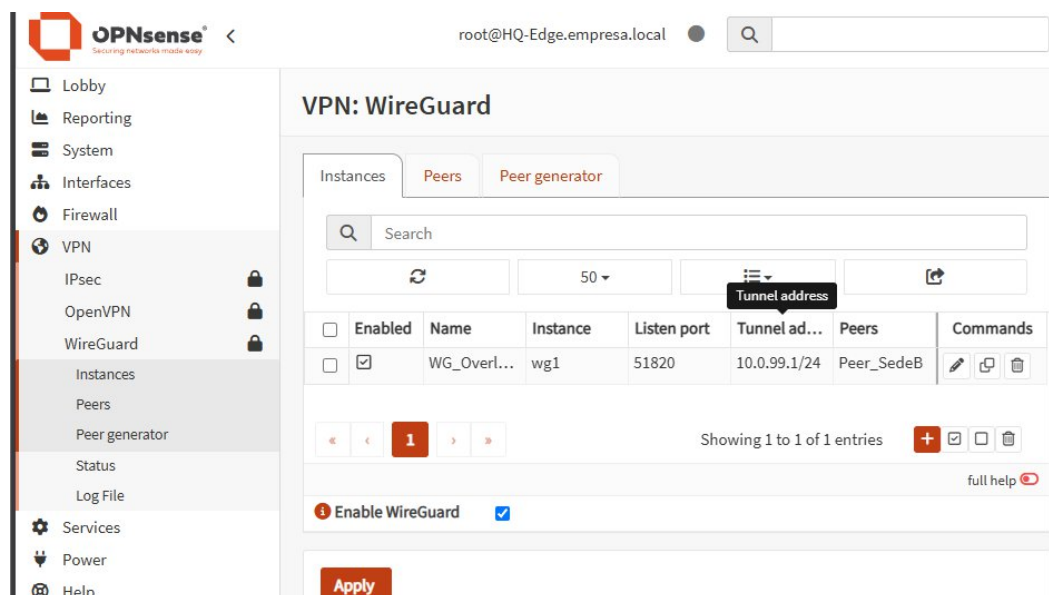
Following the automated execution of the native REST API playbook, kernel-level cryptographic sessions are established between the edge gateways.

3.1.3.1 Peer Session State Audit (wg show)

Executing diagnostic calls on the **HQ_Firewall** confirms active, bidirectional Noise protocol handshakes with **SedeB_Firewall**:

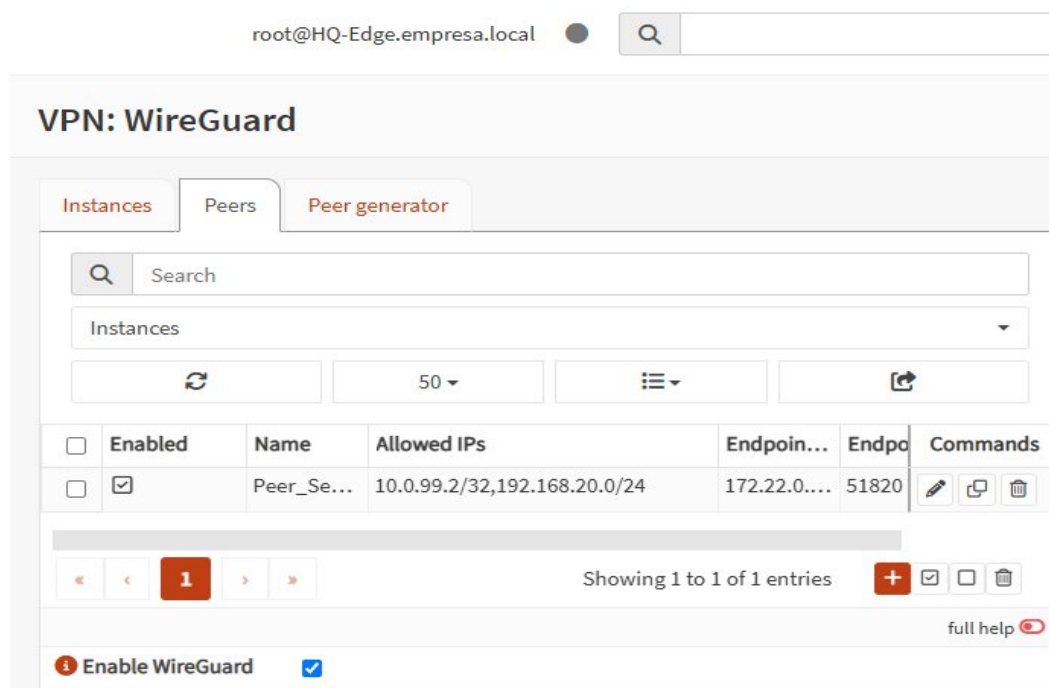
- **Cryptographic Integrity:** The latest handshake: 8 seconds ago state confirms successful DH key exchanges using Curve25519.
- **Traffic Accounting:** Non-zero byte transfer counters (180 B received, 272 B sent) prove active encapsulation and payload delivery across the 172.22.0.0/24 backbone.

In this image, when we execute the playbook with ansible, Opnsense created the configuration, but, it is important, all things is not possible with the Api key of this Firewall.



The screenshot shows the Opnsense web interface. The left sidebar contains a menu with items: Lobby, Reporting, System, Interfaces, Firewall, VPN, IPsec, OpenVPN, WireGuard, Instances, Peers, Peer generator, Status, Log File, Services, Power, and Help. The main content area is titled "VPN: WireGuard" and has three tabs: "Instances", "Peers", and "Peer generator". The "Instances" tab is active. It features a search bar, a refresh button, a dropdown menu set to "50", and a "Tunnel address" dropdown. Below these is a table with the following columns: Enabled, Name, Instance, Listen port, Tunnel ad..., Peers, and Commands. There is one entry in the table: a checked "Enabled" checkbox, "WG_Overl...", "wg1", "51820", "10.0.99.1/24", "Peer_SedeB", and icons for edit, copy, and delete. At the bottom of the table, it says "Showing 1 to 1 of 1 entries". Below the table is a checkbox labeled "Enable WireGuard" which is checked, and an "Apply" button.

Here, we have got the Peer or neighbour of the other site of this infrastructure site to site.



The screenshot shows the same Opnsense web interface, but the "Peers" tab is now active. It features a search bar, a dropdown menu set to "Instances", and buttons for refresh, dropdown (set to "50"), list view, and share. Below these is a table with the following columns: Enabled, Name, Allowed IPs, Endpoin..., Endpo, and Commands. There is one entry in the table: a checked "Enabled" checkbox, "Peer_Se...", "10.0.99.2/32,192.168.20.0/24", "172.22.0....", "51820", and icons for edit, copy, and delete. At the bottom of the table, it says "Showing 1 to 1 of 1 entries". Below the table is a checkbox labeled "Enable WireGuard" which is checked, and a "full help" link.

3.1.3.2. Interface Assignment & Kernel Binding

To ensure persistence, proper routing table injection, and stateful inspection through the OPNsense packet filter engine (pf), the virtual WireGuard tunnel interfaces (wg0 / wg1) were formally bound and enabled within the system assignments manager (Interfaces -> Assignments).

This step, of course it's good checked the assignment of interfaces. In both firewalls, HQ Firewall and B, if we have more sites, we will do the same.

Interfaces: Assignments

Interface	Identifier ?	Device	
[LAN]	lan	em1 (00:0c:29:ff:61:7c) ▾	
[WAN]	wan	em0 (00:0c:29:ff:61:68) ▾	
[WG_Tunnel]	opt1	wg1 (WireGuard - WG_Overlay_HQ) ▾	

Save

root@SedeB.internal ● 🔍

VPN: WireGuard

Instances

Peers

Peer generator

🔍 Search

🔄 50 ▾ ⋮ 📄

☐	Enabled	Name	Instance	Listen port	Tunnel ad...	Peers	Commands
☐	☒	WG_Overlay	wg0	51820	10.0.99.2/32	Peer_HQ_...	

⏪ ⏩ 1 ⏪ ⏩ Showing 1 to 1 of 1 entries

+ ☑ ☐ 🗑

full help 🗉

🔔 Enable WireGuard ☒

Apply

3.1.3.3. Inter-Site ICMP Reachability Validation

Bidirectional Layer 3 connectivity was verified by executing low-level diagnostic echo requests across the WireGuard transit segment:

- **HQ to Sede B Transit Verification and vice versa:**

```
root@HQ-Edge:~ # ping 10.0.99.2
PING 10.0.99.2 (10.0.99.2): 56 data bytes
64 bytes from 10.0.99.2: icmp_seq=0 ttl=64 time=1.089 ms
64 bytes from 10.0.99.2: icmp_seq=1 ttl=64 time=1.487 ms
^C
--- 10.0.99.2 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.089/1.288/1.487/0.199 ms
root@HQ-Edge:~ # wg show
interface: wg1
  public key: sY49j+Nn1uspjZ/X8Gqn5agxxg4MoZGardZMoELgvWs=
  private key: (hidden)
  listening port: 51820
peer: 0fC2NNY79ZKKuWcaEeUx9fqEk8bhcfXoLFqgPGv/ais=
  endpoint: 172.22.0.188:51820
  allowed ips: 192.168.20.0/24, 10.0.99.2/32
  latest handshake: 56 seconds ago
  transfer: 3.54 MiB received, 21.81 KiB sent
  persistent keepalive: every 25 seconds
root@HQ-Edge:~ #

root@SedeB:~ # ping 10.0.99.1
PING 10.0.99.1 (10.0.99.1): 56 data bytes
64 bytes from 10.0.99.1: icmp_seq=0 ttl=64 time=1.104 ms
64 bytes from 10.0.99.1: icmp_seq=1 ttl=64 time=1.018 ms
64 bytes from 10.0.99.1: icmp_seq=2 ttl=64 time=0.951 ms
^C
--- 10.0.99.1 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.951/1.024/1.104/0.063 ms
root@SedeB:~ # wg show
interface: wg0
  public key: 0fC2NNY79ZKKuWcaEeUx9fqEk8bhcfXoLFqgPGv/ais=
  private key: (hidden)
  listening port: 51820
peer: sY49j+Nn1uspjZ/X8Gqn5agxxg4MoZGardZMoELgvWs=
  endpoint: 172.22.0.187:51820
  allowed ips: 10.168.10.0/24, 10.0.99.1/32
  latest handshake: 1 minute, 1 second ago
  transfer: 17.40 MiB received, 3.91 KiB sent
  persistent keepalive: every 25 seconds
root@SedeB:~ #
```

Step 4. Dynamic Routing & Policy Enforcement (HQ ↔ Sede B)

El objetivo de esta fase es automatizar el intercambio de rutas entre el **HQ_Firewall** y la **SedeB_Firewall** utilizando **BGP (Border Gateway Protocol)** sobre el túnel WireGuard previamente establecido (10.0.99.1 y 10.0.99.2), y posteriormente aplicar reglas de cortafuegos (*Packet Filter*) para controlar el tráfico entre ambas sedes.

4.1 Dynamic Routing Engine (FRRouting / BGP Implementation)

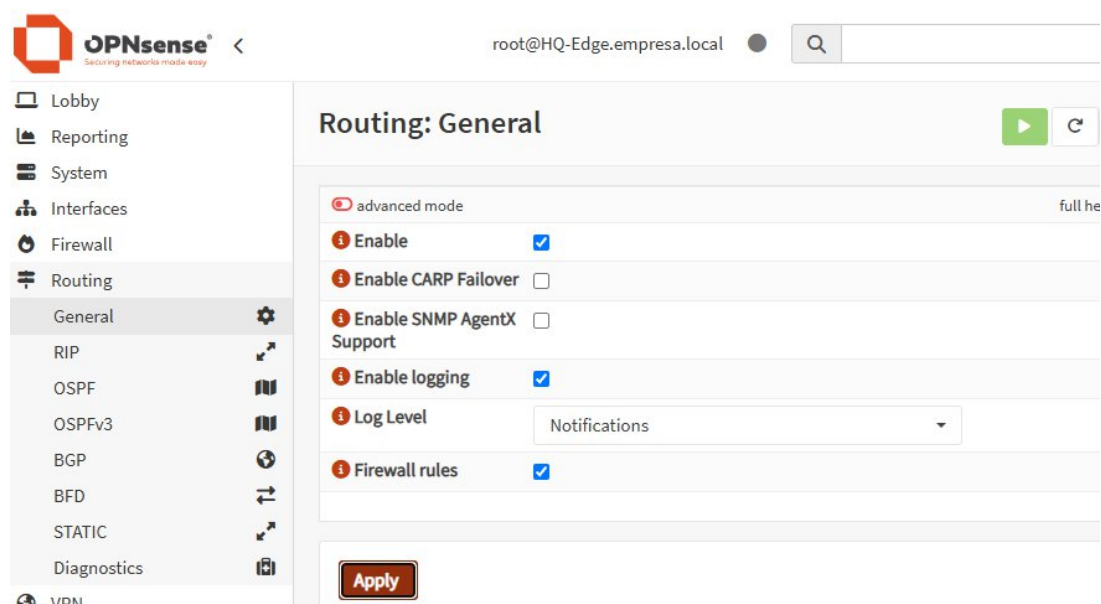
4.1.1 BGP Autonomous System (AS) Allocation & Peering Matrix

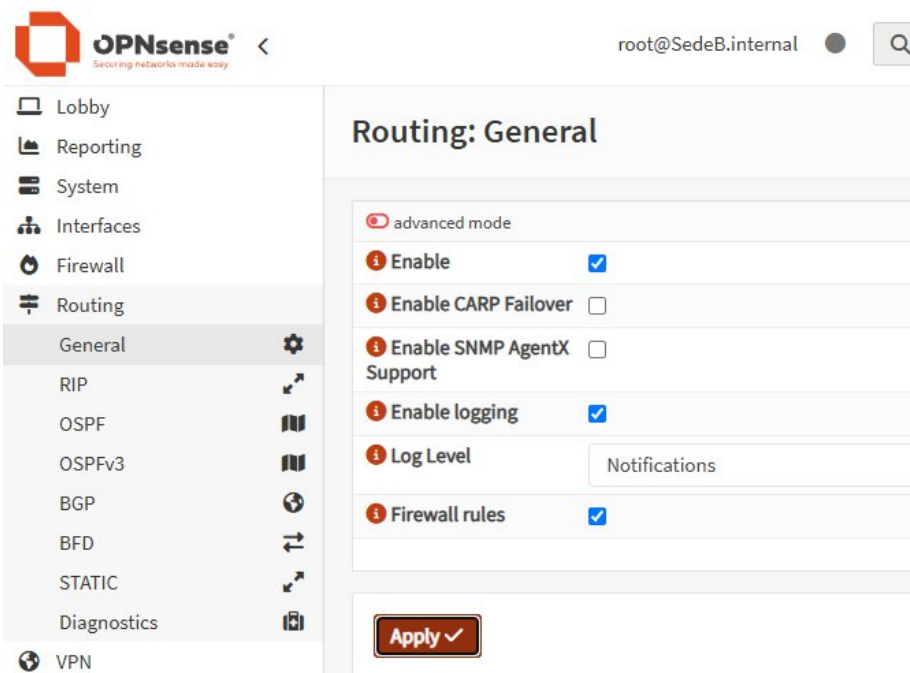
Se establece un esquema **eBGP** (External BGP) entre la oficina central y la rama B:

Node Hostname	Role	BGP Autonomous System (ASN)	WireGuard Overlay IP	Subnet Anunciada
HQ_Firewall	Central Hub	AS 65001	10.0.99.1/24	10.168.10.0/24
SedeB_Firewall	Spoke B	AS 65002	10.0.99.2/24	192.168.20.0/24

- **Configuración del Plugin os-frr en OPNsense:**
 - **En HQ_Firewall:**
 - BGP Router ID: 10.0.99.1
 - Autonomous System: 65001
 - Neighbor: 10.0.99.2 (Remote AS: 65002)
 - Network Prefix: 10.168.10.0/24
 - **En SedeB_Firewall:**
 - BGP Router ID: 10.0.99.2
 - Autonomous System: 65002
 - Neighbor: 10.0.99.1 (Remote AS: 65001)
 - Network Prefix: 192.168.20.0/24

In this picture, we must enable the service of Routing in Opnsense. (HQ Firewall and SedeB)





```

root@HQ-Edge:~ # service frr restart
Stopping watchfrr.
Waiting for PIDS: 354.
Starting watchfrr.
2026/07/21 20:09:03 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:03 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:03 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:03 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:03 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf
RCVBUF set to 2097152 (requested 16777216)
[76363!mgmtd] sending configuration
[77198!zebrd] sending configuration
[79207!bgpd] sending configuration
Waiting for children to finish applying config...
[85686!watchfrr] sending configuration
[76363!mgmtd] done
[77198!zebrd] done
[79207!bgpd] done
[85686!watchfrr] done
root@HQ-Edge:~ #

```

This command in this picture, we can see, we haven't got any neighbour, but this is temporarily, in next steps, we will have the neighbour.

```

root@HQ-Edge:~ # vtysh -c 'show ip bgp summary'
2026/07/21 20:09:28 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf: fd 3: SO_
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:28 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf: fd 4: SO_
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:28 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf: fd 5: SO_
RCVBUF set to 2097152 (requested 16777216)
2026/07/21 20:09:28 [H0DHT-S9KF21][EC 100663299] setsockopt_so_rcvbuf: fd 6: SO_
RCVBUF set to 2097152 (requested 16777216)
% No BGP neighbors found in VRF default
root@HQ-Edge:~ #

```

With a Ansible Playbook, we can deploy the BGP in our Wireguard

```
ruben@adminsyst:~/proyecto$ ansible-playbook -i inventory/host.ini deploy_bgp.yml --ask
-vault-pass
Vault password:

PLAY [Step 4.1 - Dynamic eBGP Provisioning via FRR API] *****

TASK [Enable FRR General Service] *****
ok: [SedeB_Firewall]
ok: [HQ_Firewall]

TASK [Configure BGP General Settings & Local Network] *****
ok: [HQ_Firewall]
ok: [SedeB_Firewall]

TASK [Add BGP Neighbor Endpoint] *****
ok: [SedeB_Firewall]
ok: [HQ_Firewall]

TASK [Apply FRR Configuration & Reload Routing Engine] *****
ok: [SedeB_Firewall]
ok: [HQ_Firewall]

PLAY RECAP *****
HQ_Firewall           : ok=4    changed=0    unreachable=0    failed=0    skipped
=0    rescued=0    ignored=0
SedeB_Firewall        : ok=4    changed=0    unreachable=0    failed=0    skipped
=0    rescued=0    ignored=0
```

4.1.2 BGP Session Establishment & Peering Audit

Verificación del estado del vecino BGP (*Established*) utilizando la shell interactiva de FRRouting (vtysh):

Finally, we have connected both sites, HQ Firewall and SedeB.

```
IPv4 Unicast Summary:
BGP router identifier 10.0.99.1, local AS number 65551 VRF default vrf-id 0
BGP table version 3
RIB entries 1, using 152 bytes of memory
Peers 1, using 17 KiB of memory

Neighbor      U      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  Sta
te/PfxRcd    PfxSnt Desc
10.0.99.2      4      65002      5        10        0     0     0      never
Idle          0 Peering Sede B

Total number of neighbors 1
root@HQ-Edge:~ #
```

BGP learned the route without help. It's dynamic.

```
total number of neighbors 1
HQ-Edge.empresa.local# show ip bgp
BGP table version is 2, local router ID is 10.0.99.1, vrf id 0
Default local pref 100, local AS 65001
Status codes: s suppressed, d damped, h history, u unsorted, * valid, > best, =
multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes: i - IGP, e - EGP, ? - incomplete
RPKI validation codes: U valid, I invalid, N Not found

   Network          Next Hop             Metric LocPrf Weight Path
*>  10.168.10.0/24   0.0.0.0                  0         32768 i
*>  192.168.20.0/24 10.0.99.2                  0         0 65002 i

Displayed 2 routes and 2 total paths
HQ-Edge.empresa.local#
```

Output Analysis: - Local Network: 10.168.10.0/24 originates locally (Next Hop 0.0.0.0) with Weight 32768. - Learned Overlay Route: 192.168.20.0/24 is dynamically learned from peer 10.0.99.2 via AS 65002. - Path Selection: Both routes exhibit the valid (*) and best (>) path flags, indicating successful installation into the active kernel Routing Information Base (RIB).

4.1.3 Route Propagation & Kernel Inspection

Verification that the Remote Site B network (192.168.20.0/24) has been successfully injected into the FreeBSD kernel Routing Information Base (RIB) on OPNsense-HQ with the `B` (BGP) routing protocol flag:

Bash

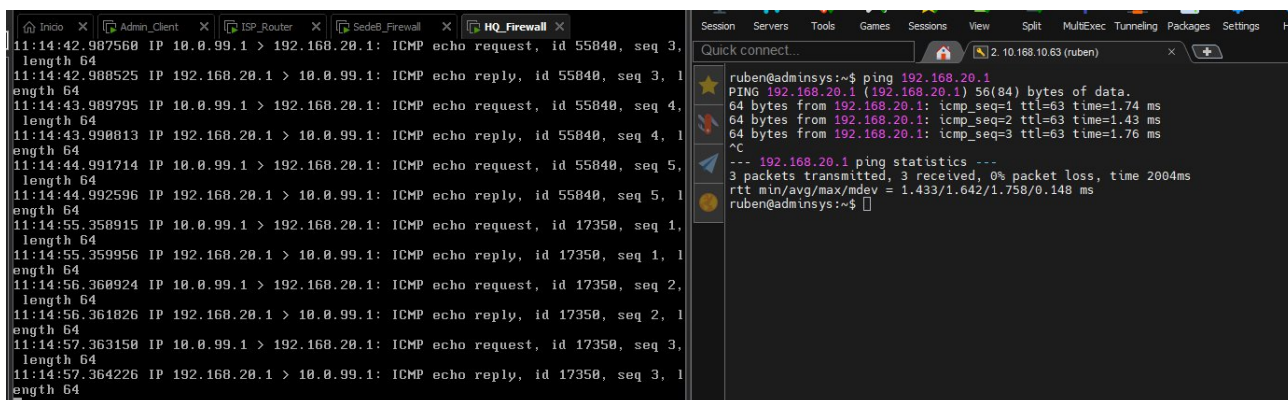
netstat -rn -f inet

```
root@HQ-Edge:~ # netstat -rn -f inet
Routing tables

Internet:
Destination          Gateway             Flags              Netif  Expire
default              172.22.0.1         UGS                em0
10.0.99.0/24         link#7             U                  wg1
10.0.99.1            link#3             UHS                lo0
10.0.99.2            link#7             UHS                wg1
10.168.10.0/24       link#2             U                  em1
10.168.10.1          link#3             UHS                lo0
127.0.0.1            link#3             UH                 lo0
172.22.0.0/24        link#1             U                  em0
172.22.0.187         link#3             UHS                lo0
192.168.20.0/24      link#7             US                 wg1
root@HQ-Edge:~ #
```

4.2 Site-to-Site Inter-LAN Firewall Policy Enforcement

Once dynamic route propagation is established, inter-site traffic between the Central Site LAN (10.168.10.0/24) and the Remote Branch LAN (192.168.20.0/24) must be filtered strictly adhering to the Principle of Least Privilege (PoLP). By default, an implicit deny rule drops all unauthorized inter-LAN traffic across the WireGuard tunnel. Explicit stateful firewall rules are provisioned to permit only required administrative, operational, and management services.



The screenshot shows a terminal window with two panes. The left pane displays a packet capture from the HQ_Firewall interface, showing ICMP echo requests and replies between 10.0.99.1 and 192.168.20.1. The right pane shows the output of a ping command from a management client (10.168.10.63) to the remote site gateway (192.168.20.1). The ping statistics show 3 packets transmitted, 3 received, 0% packet loss, and a round-trip time of approximately 1.43ms to 1.76ms.

```
11:14:42.987568 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 55840, seq 3, length 64
11:14:42.988525 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 55840, seq 3, length 64
11:14:43.989795 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 55840, seq 4, length 64
11:14:43.990813 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 55840, seq 4, length 64
11:14:44.991714 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 55840, seq 5, length 64
11:14:44.992596 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 55840, seq 5, length 64
11:14:55.358915 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 17350, seq 1, length 64
11:14:55.359956 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 17350, seq 1, length 64
11:14:56.360924 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 17350, seq 2, length 64
11:14:56.361826 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 17350, seq 2, length 64
11:14:57.363150 IP 10.0.99.1 > 192.168.20.1: ICMP echo request, id 17350, seq 3, length 64
11:14:57.364226 IP 192.168.20.1 > 10.0.99.1: ICMP echo reply, id 17350, seq 3, length 64

ruben@adminsinsys:~$ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data:
64 bytes from 192.168.20.1: icmp_seq=1 ttl=63 time=1.74 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=63 time=1.43 ms
64 bytes from 192.168.20.1: icmp_seq=3 ttl=63 time=1.76 ms
^C
--- 192.168.20.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.433/1.642/1.758/0.148 ms
ruben@adminsinsys:~$
```

1. Packet Capture on HQ Firewall (HQ_Firewall) tcpdump -ni wg1 icmp
2. Traffic Generation from Management Client (10.168.10.63) ping 192.168.20.1 Output Analysis: - Source Host: 10.168.10.63 (Central LAN Management Node) - Target Gateway: 192.168.20.1 (Remote Site B Gateway) - Traffic Inspection: Bidirectional flow confirmed with matching ICMP echo request and echo reply packets traversing wg1 without packet loss (0% loss, ~1.6ms average latency).

4.2.1 Allowed Services & Inter-Site Traffic Matrix

Inter-site traffic across the WireGuard virtual interface is restricted to essential services using explicit stateful firewall rules, enforcing the Principle of Least Privilege (PoLP):

- **LDAPS / AD Services:** Permits inbound traffic from Remote Site B (192.168.20.0/24) to the HQ Domain Controller (HQ_ADDS, 10.168.10.2) for LDAPS (TCP 636), DNS (TCP/UDP 53), and Kerberos (TCP/UDP 88).
- **Admin Management:** Permits SSH (TCP 22) and HTTPS (TCP 443) traffic originating from the management node (Admin_Client, 10.168.10.0/24) toward Remote Site B LAN and gateway interfaces.
- **Default Deny (POLP-03):** Explicitly drops all unlisted inter-site traffic to maintain a Zero-Trust baseline.

```

ruben@adminsyst:~/proyecto$ ansible-playbook -i inventory/host.ini fire.yml --ask-vault-pass
Vault password:

PLAY [Apply Strict PoLP Firewall Rules (No ICMP)] *****

TASK [HQ | Allow AD Services from Remote Branch (192.168.20.0/24 -> 10.168.10.X)] *****
skipping: [SedeB_Firewall]
ok: [HQ_Firewall]

TASK [SedeB | Allow Admin Client Access (SSH / HTTPS)] *****
skipping: [HQ_Firewall]
ok: [SedeB_Firewall]

TASK [ALL | Apply Firewall Filter Rules] *****
ok: [SedeB_Firewall]
ok: [HQ_Firewall]

PLAY RECAP *****
HQ_Firewall      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored
SedeB_Firewall   : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored
ruben@adminsyst:~/proyecto$

```

☐	L	☒		WireGuard (Group)	IPv4	TCP/UDP	192.168.20.0/24	*	10.168.10.0/24	*	*	POLP-01: Allow DNS from Sede B		
☐	L	☒		WireGuard (Group)	IPv4	TCP/UDP	192.168.20.0/24	*	10.168.10.0/24	*	*	POLP-01: Allow Kerberos from Sede B		
☐	L	☒		WireGuard (Group)	IPv4	TCP/UDP	192.168.20.0/24	*	10.168.10.0/24	*	*	POLP-01: Allow LDAPS from Sede B		
☐	L	☒		WireGuard (Group)	IPv4	*	*	*	*	*	*	POLP-04: Explicit Default Deny Inter-Site Traffic		

Successful TCP handshake verification for explicitly allowed Active Directory services (LDAPS/636, DNS/53, Kerberos/88) from Remote Site B to HQ Domain Controller (10.168.10.2).

```

ruben@adminsyst:~/proyecto$ nc -zv 10.168.10.2 636
Connection to 10.168.10.2 636 port [tcp/ldaps] succeeded!
ruben@adminsyst:~/proyecto$ nc -zv 10.168.10.2 53
Connection to 10.168.10.2 53 port [tcp/domain] succeeded!
ruben@adminsyst:~/proyecto$ nc -zv 10.168.10.2 88
Connection to 10.168.10.2 88 port [tcp/kerberos] succeeded!
ruben@adminsyst:~/proyecto$

```

Verification of the explicit Default Deny policy enforcement. Unpermitted traffic (HTTP/80) is silently dropped by the OPNsense gateway, resulting in a connection timeout.

```

ruben@adminsyst:~/proyecto$ nc -zv 10.168.10.2 80

```

Step 5. End-to-End Enterprise Integration & Final Acceptance Testing

To conclusively validate the production-readiness of the multi-site infrastructure, a end-to-end integration scenario was executed using a standard Windows Workstation deployed within the isolated **Remote Branch B LAN (192.168.20.0/24)**.

This final test suite validates the simultaneous operation of all architectural layers:

Outbound WAN Transit (NAT), Encrypted Cross-Site Overlay Routing (WireGuard/BGP), and Centralized Identity Management (Active Directory Domain Services).

```
C:\Users\prueba>ping 8.8.8.8

Haciendo ping a 8.8.8.8 con 32 bytes de datos:
Respuesta desde 8.8.8.8: bytes=32 tiempo=12ms TTL=126
Respuesta desde 8.8.8.8: bytes=32 tiempo=12ms TTL=126
Respuesta desde 8.8.8.8: bytes=32 tiempo=12ms TTL=126
Respuesta desde 8.8.8.8: bytes=32 tiempo=20ms TTL=126

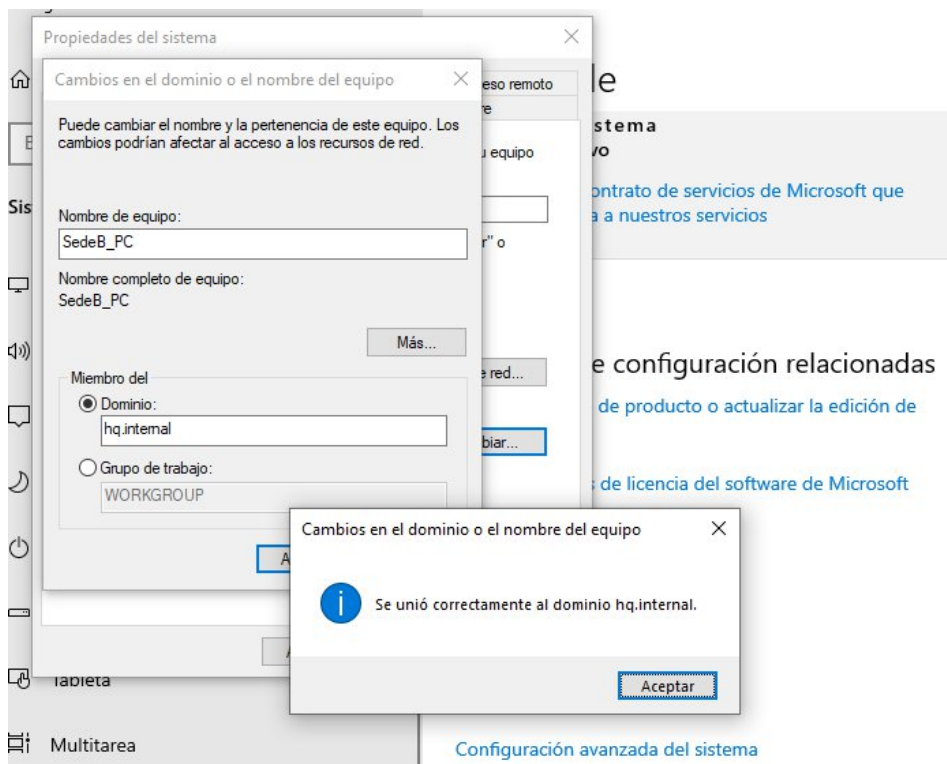
Estadísticas de ping para 8.8.8.8:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
        (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 12ms, Máximo = 20ms, Media = 14ms

C:\Users\prueba>
```

Expected Result: Successful resolution returning the primary Domain Controller IP (10.168.10.2), verifying that DNS traffic successfully traverses the WireGuard overlay under rule POLP-01

```
C:\Users\prueba>nslookup hq.internal
Servidor:  hqinternal.hq.internal
Address:  10.168.10.2

Nombre:   hq.internal
Address:  10.168.10.2
```



```
CA. Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Ruper>whoami
hq\ruper

C:\Users\Ruper>
```

CONCLUSION

This project successfully demonstrates a robust, automated multi-site network infrastructure. By combining WireGuard for encrypted site-to-site communication, eBGP for dynamic route propagation, and Ansible for IaC-driven firewall orchestration, the architecture provides a highly secure, scalable, and fully reproducible environment aligned with industry best practices.